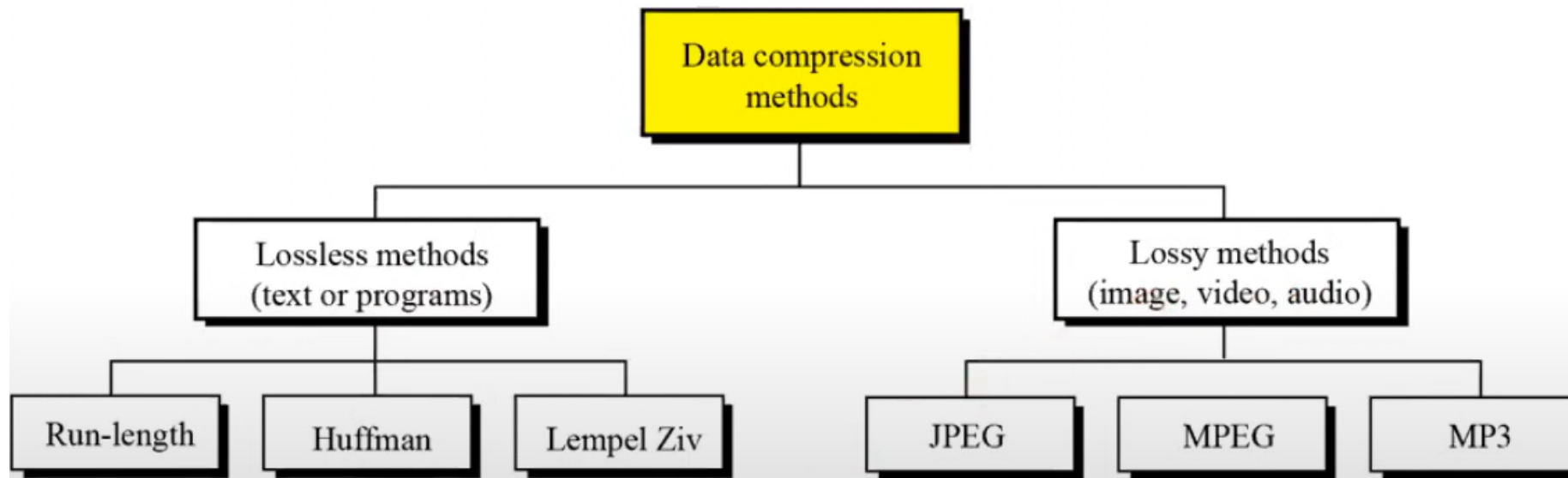


Xử lý ảnh và Thị giác máy tính

Chương 4. Nén ảnh và phân đoạn ảnh

- ❑ Data compression
 - ❑ Lossless methods
 - ❑ Lossy methods



PHẦN 1. NÉN ẢNH

1. Nén dữ liệu không tổn thất (lossless)

- ❑ Dữ liệu được bảo toàn, nhưng tỉ lệ nén thấp

Các kiểu mã hóa thông dụng trong lossless compression

- ❑ Run-length coding (RLC)
- ❑ Huffman coding
- ❑ Lempel-Ziv-Welch

Run-length coding

❑ Tận dụng tính chất lặp lại giá trị trong ảnh

❑ Nguyên tắc:

Dùng cặp giá trị (**độ dài chuỗi**, **giá trị**) để thay thế cho loạt dữ liệu lặp lại

Ví dụ:

5	5	5	5	5	5	7	7	7	7	10	10	10
↙						↓				↘		
(6, 5)						(4, 7)				(3, 10)		

* Chú ý nếu chiều dài chuỗi lặp > 255 thì 1 byte sẽ không đủ, khi đó tách ra hai chuỗi, một chuỗi có chiều dài 255, chuỗi kia chứa phần còn lại

Giải mã (Run-length decoding)

- ❑ Lấy từng cặp giá trị (**độ dài chuỗi**, **giá trị**)
- ❑ Tái tạo lại chuỗi ban đầu bằng cách lặp lại **giá trị** tương ứng với **số lần** chỉ định

Ví dụ:

(6 , 5)	→	5 5 5 5 5 5
(4 , 7)	→	7 7 7 7
(3 , 10)	→	10 10 10



5 5 5 5 5 5 7 7 7 7 10 10 10

Ưu điểm

- ❑ Đơn giản, dễ triển khai
- ❑ Hiệu quả với những dạng dữ liệu có tính lặp lại cao.

Ứng dụng:

- ❑ Nén ảnh đơn sắc (nhị phân), ảnh có vùng màu đồng nhất (.BMP, .TIF)
- ❑ Nén tập tin văn bản, mã vạch máy in

Nhược điểm

- ❑ RLC chỉ đạt được hiệu quả khi chiều dài chuỗi lặp lớn hơn một ngưỡng nhất định (≥ 3), khi dữ liệu đơn lẻ xuất hiện nhiều RLC sẽ giảm hiệu quả

Ví dụ:

2 4 6 8 10 12 $\rightarrow$ (1, 2) (1, 4) (1, 6) (1, 8) (1, 10) (1, 12)

Tỉ lệ nén: $6/12 = 0.5$

Run-length coding cải tiến (1)

- ❑ Chỉ mã hoá độ dài loạt dữ liệu lặp lại
- ❑ Thêm kí tự tiền tố vào trước độ dài loạt

Ví dụ:

4 7 3 7 7 7 7 7 7 7 9 9 9 9 9 9

→ 4 7 3 + 8 7 + 6 9

Tỉ lệ nén: $17/9 = 1.89$

Run-length coding cải tiến (2)

❑ Kiểm tra mức độ tương quan giữa các dữ liệu kề nhau

Ví dụ:

5 7 9 11 13 18 28 38 48 58 55 60 65 70 75 80 85

→ 5 2 2 2 2 5 10 10 10 10 -3 5 5 5 5 5 5

→ (1, 5) (4, 2) (1, 5) (4, 10) (1, -3) (6, 5)

Tỉ số nén $17/12 = 1.42$ (nếu 1 số tương ứng 1 byte)

Thuật toán RLC cho ảnh

- ❑ Tiến hành duyệt trên từng hàng cho đến khi kết thúc vùng dữ liệu ảnh
- ❑ Đối với ảnh chiều dài loạt cực đại chính là kích thước của hàng (row)

Ví dụ:

Row 1	255	255	255	0	0	255	255	255
Row 2	0	0	255	255	255	255	255	255
Row 3	255	255	0	0	0	255	255	255



Row 1: (3, 255) (2, 0) (3, 255)

Row 2: (2, 0) (6, 255)

Row 3: (2, 255) (3, 0) (3, 255)

Mã hóa Huffman (encoding)

Thuật toán

Bước 1. Liệt kê các phần tử dưới dạng tần suất xuất hiện, sắp xếp bảng dưới dạng tăng dần

Bước 2. Kết hợp 2 phần tử có tần suất thấp nhất lại thành 1 phần tử mới. Cập nhật lại bảng (loại bỏ 2 phần tử đó)

Bước 3. Lặp lại bước 2 cho đến khi bảng chỉ còn 1 phần tử (tạo cây Huffman)

Bước 4. Từ gốc (phần tử có tần suất lớn nhất) đi xuống, mỗi lần xuống **bên phải** thêm 1 bit "**1**", **bên trái** thêm bit "**0**"

Chú ý

- ❑ Đánh dấu **0** hay **1** cho nhánh sẽ không ảnh hưởng đến việc giải mã
- ❑ Do chúng ta chỉ quan tâm đến số lượng bit giảm xuống bao nhiêu, tức số bit như nhau dù đánh dấu mã khác nhau

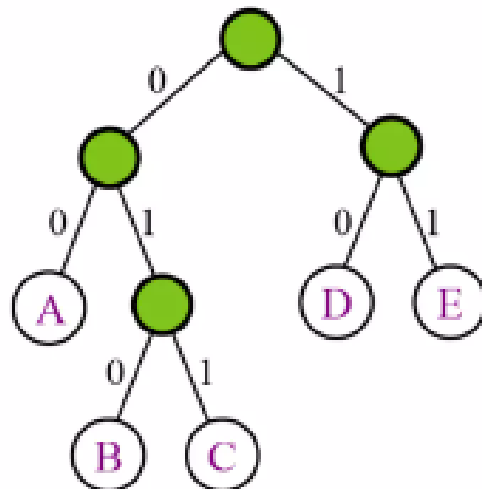
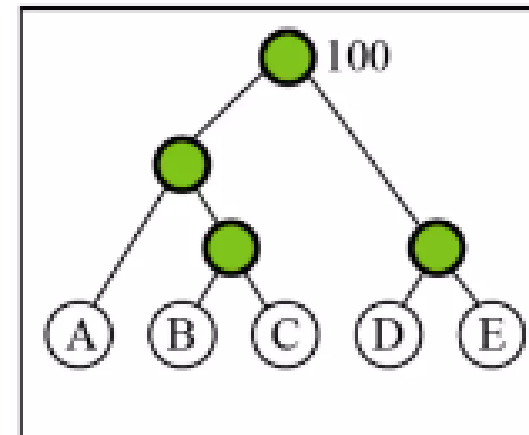
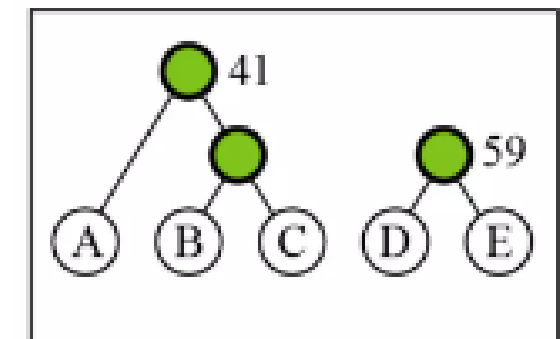
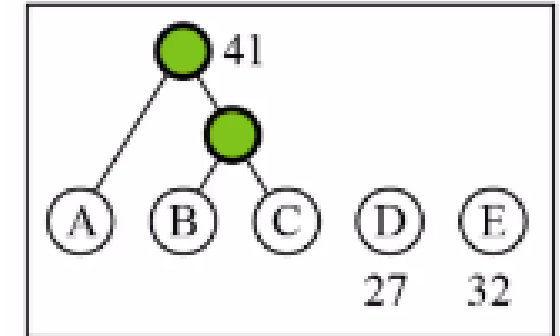
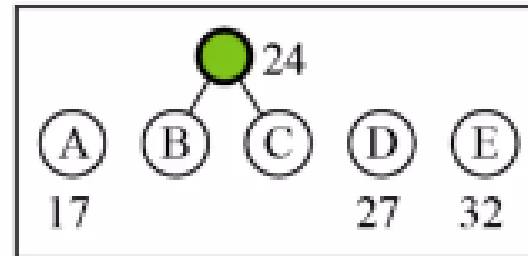
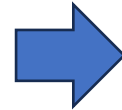
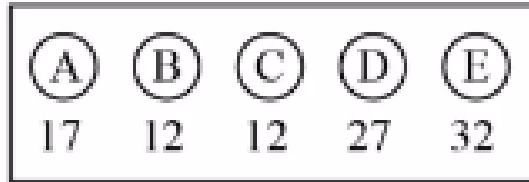
Ví dụ Mã hóa Huffman

- ❑ Xây dựng cây Huffman từ bảng thống kê tần suất xuất hiện sau:

Character	A	B	C	D	E
Frequency	17	12	12	27	32

Ví dụ

Character	A	B	C	D	E
Frequency	17	12	12	27	32



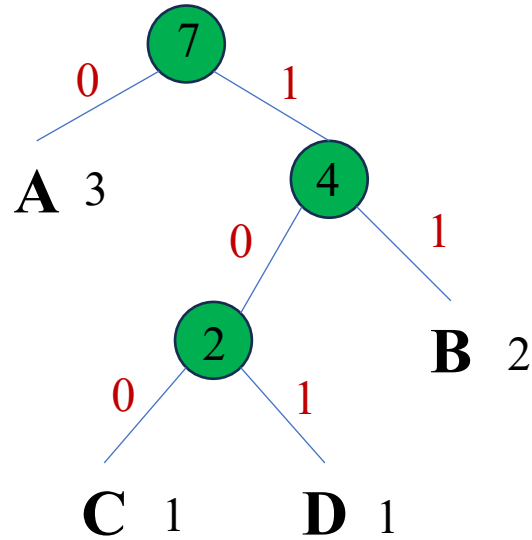
A: 00	D: 10
B: 010	E: 11
C: 011	

Code

Ví dụ Mã hóa Huffman

Mã hóa dùng mã Huffman

□ AABCBAD



Code	
A	0
B	11
C	100
D	101



AABCBAD



0 0 11 100 11 0 101

Huffman code

Giải mã (decoding)

- ❑ Trong phương pháp mã Huffman mã của ký tự là duy nhất và không mã nào là phần bắt đầu của mã trước
- ❑ Việc giải mã chắc chắn phải sử dụng cây nhị phân giống như trong mã hoá.

Giải mã (decoding)

- ❑ Dùng cây Huffman đã xây dựng ở mã hóa
- ❑ Đọc từng bit của chuỗi nhị phân, bắt đầu từ gốc
- ❑ Mỗi lần đọc đến một nút lá, ngừng ghi nhận ký tự tương ứng
- ❑ Tiếp tục đọc bit còn lại (làm giống từ bước 2).

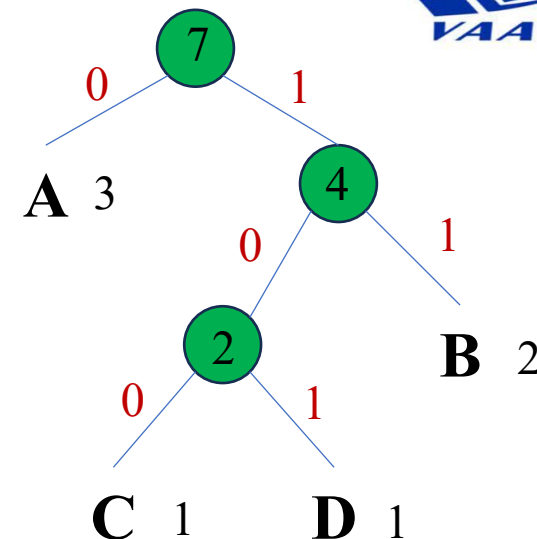
Ví dụ Giải mã (decoding)



Huffman code

0 0 11 100 11 0 101

1. Đọc bit **0** (đầu tiên)
Đi nhánh trái (là 0) → Gặp nút Lá (ngừng): **A**
2. Đọc bit tiếp theo (**0**)
Đi nhánh trái (là 0) → Gặp nút Lá (ngừng): **A**
3. Đọc bit tiếp theo (**11**)
Đi nhánh phải (là 1) → Nhánh phải (là 1) → Gặp nút Lá (ngừng): **B**
4. Đọc bit tiếp theo (**100**)
Đi nhánh phải (là 1) → Nhánh trái (là 0) → Nhánh trái (là 0) → Gặp nút Lá (ngừng): **C**
5. Đọc bit tiếp theo (**11**)
Đi nhánh phải (là 1) → Nhánh phải (là 1) → Gặp nút Lá (ngừng): **B**
6. Đọc bit tiếp theo (**0**)
Đi nhánh trái (là 0) → Gặp nút Lá (ngừng): **A**
7. Đọc bit tiếp theo (**101**)
Đi nhánh phải (là 1) → Nhánh trái (là 0) → Nhánh phải (là 1) → Gặp nút Lá (ngừng): **D**



Chuỗi được giải mã → **AABCBAD**

Ví dụ Mã hóa Huffman

Mã hóa dùng mã Huffman

□ AAAAAABCCCCCCDDEEEEE

Mã hóa Huffman trong xử lý ảnh

- ❑ Thực hiện mã hóa ảnh sau dùng thuật toán Huffman.
Biết ảnh được chia theo các khối 2x2 (mỗi khối sẽ như là một chữ cái của bức ảnh)

$$I = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 1 \\ \hline 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ \hline 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix}$$

Mã hóa Huffman

Ưu điểm:

- ☐ Hiệu quả nén tốt, đặc biệt là khi dữ liệu có tần suất xuất hiện chênh lệch.
- ☐ Dễ hiểu và dễ triển khai

Nhược điểm:

- ☐ Chi phí cao tính toán tần suất và xây dựng cây Huffman
- ☐ Khi dữ liệu có tần suất phân bố đồng đều nhau

Ứng dụng:

- ☐ Nén ảnh (JPEG, TIFF), âm thanh (MP3)
- ☐ Nén văn bản (ZIP), tập tin và cơ sở dữ liệu

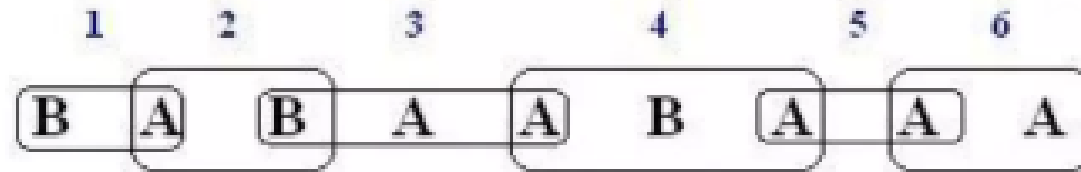
Mã hóa LZW

- ☐ Lempel, Ziv, Welch
- ☐ Dùng một bộ từ điển tự động cập nhật và đánh số khi có mã mới xuất hiện
- ☐ Mã từ 0 đến 255 miêu tả 1 ký tự 8-bit
- ☐ Mã từ 256 đến 4095 được tạo và cập nhật khi xuất hiện chuỗi ký tự mới
- ☐ Tại mỗi bước, một byte được nhập vào 1 chuỗi cho đến khi chuỗi đó không tồn tại trong từ điển

Thuật toán mã hóa (encoding)

Ví dụ

BABAABAAA



1. **BA** is not in the Dictionary; insert **BA**, output the code for its prefix: **code(B)**
2. **AB** is not in the Dictionary; insert **AB**, output the code for its prefix: **code(A)**
3. **BA** is in the Dictionary.
BAA is not in Dictionary; insert **BAA**, output the code for its prefix: **code(BA)**
4. **AB** is in the Dictionary.
ABA is not in the Dictionary; insert **ABA**, output the code for its prefix: **code(AB)**
5. **AA** is not in the Dictionary; insert **AA**, output the code for its prefix: **code(A)**
6. **AA** is in the Dictionary and it is the last pattern: output its code: **code(AA)**

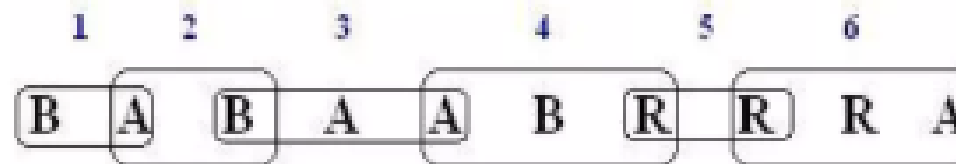
output	Dictionary	
	Code Word	String
66	256	BA
65	257	AB
256	258	BAA
257	259	ABA
65	260	AA
260		

Chuỗi được mã hóa → **<66><65><256><257><65><260>**

Ví dụ



BABAABRRRA



1. **BA** is not in the Dictionary; insert **BA**, output the code for its prefix: **code(B)**
2. **AB** is not in the Dictionary; insert **AB**, output the code for its prefix: **code(A)**
3. **BA** is in the Dictionary.
 BAA is not in Dictionary; insert **BAA**, output the code for its prefix: **code(BA)**
4. **AB** is in the Dictionary.
 ABR is not in the Dictionary; insert **ABR**, output the code for its prefix: **code(AB)**
5. **RR** is not in the Dictionary; insert **RR**, output the code for its prefix: **code(R)**
6. **RR** is in the Dictionary.
 RRA is not in the Dictionary and it is the last pattern; insert **RRA**, output code for its prefix: **code(RR)**, then output code for last character: **code(A)**

output	Dictionary	
	Code Word	String
66	256	BA
65	257	AB
256	258	BAA
257	259	ABR
82	260	RR
260	261	RRA
65		

Chuỗi được mã hóa → **<66><65><256><257><82><260><65>**

Bài tập

Dùng mã LZW để nén các chuỗi ký tự sau

- ❑ CFCFCFCCFCCFC
- ❑ ababbabababba
- ❑ !BAN!BA!BAA!BAR!

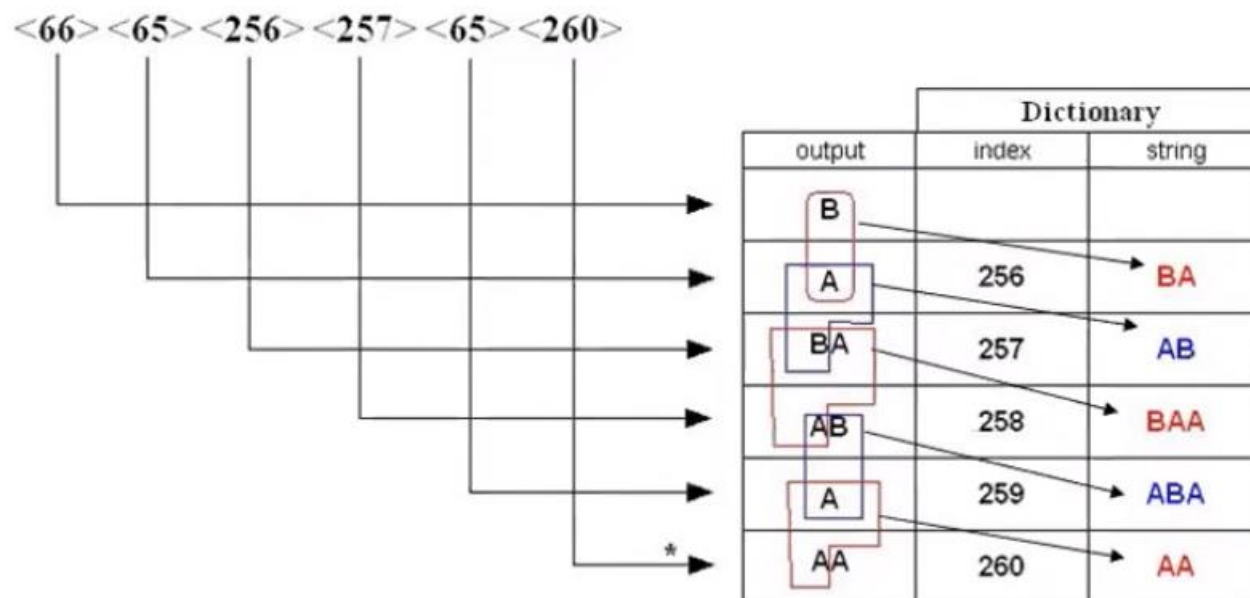
Thuật toán giải mã (decoding)



Ví dụ

Giải nén dữ liệu sau
dùng LZW

<66><65><256><257><65><260>



1. 66 is in Dictionary; output **string(66)** i.e. **B**
2. 65 is in Dictionary; output **string(65)** i.e. **A**, insert **BA**
3. 256 is in Dictionary; output **string(256)** i.e. **BA**, insert **AB**
4. 257 is in Dictionary; output **string(257)** i.e. **AB**, insert **BAA**
5. 65 is in Dictionary; output **string(65)** i.e. **A**, insert **ABA**
6. 260 is not in Dictionary; output
previous output + previous output first character: **AA**, insert **AA**

Chuỗi được giải mã → **BABAABAAA**

Bài tập

Giải nén dữ liệu sau dùng LZW

<86> <65> <65> <256> <257> <86>

<67> <70> <256> <258> <259> <257>

Bài tập

- ❑ Giả sử từ điển ban đầu chỉ có 2 bit 0 và 1 để mã hóa tương ứng a và b. Giải nén dữ liệu sau:

abababababab

Dictionary

0 a

1 b

Ứng dụng

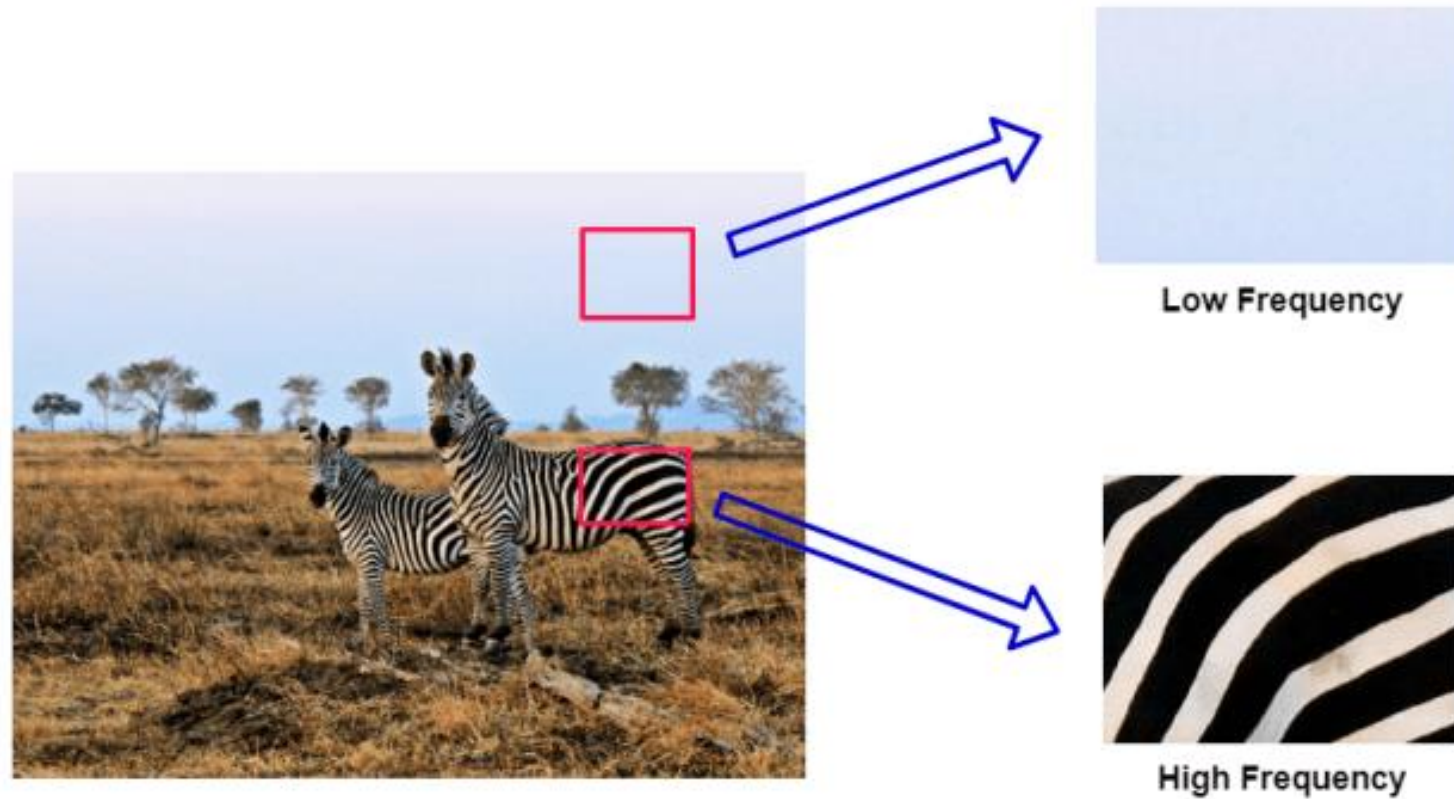
- ❑ LZW đã được sử dụng trong phần mềm nén mã nguồn mở, nó đã trở thành 1 phần không thể thiếu trong HDH UNIX CIRCA 1986
- ❑ LZW đã trở nên phổ biến khi nó được sử dụng làm 1 phần của file GIF năm 1987. Nó cũng có thể được sử dụng trong TIFF và PDF file

2. Nén có mất mát (lossy methods)

- ❑ Sau khi giải nén không thu lại dữ liệu gốc
- ❑ Tận dụng tính chất của mắt người, chấp nhận một số mất mát chấp nhận được (tương tự nhạc mp3)
- ❑ Loại bỏ chủ yếu ở các thành phần tần số cao (do mắt người khó phát hiện mất mát tại vùng tần số cao)

Chuẩn nén JPEG

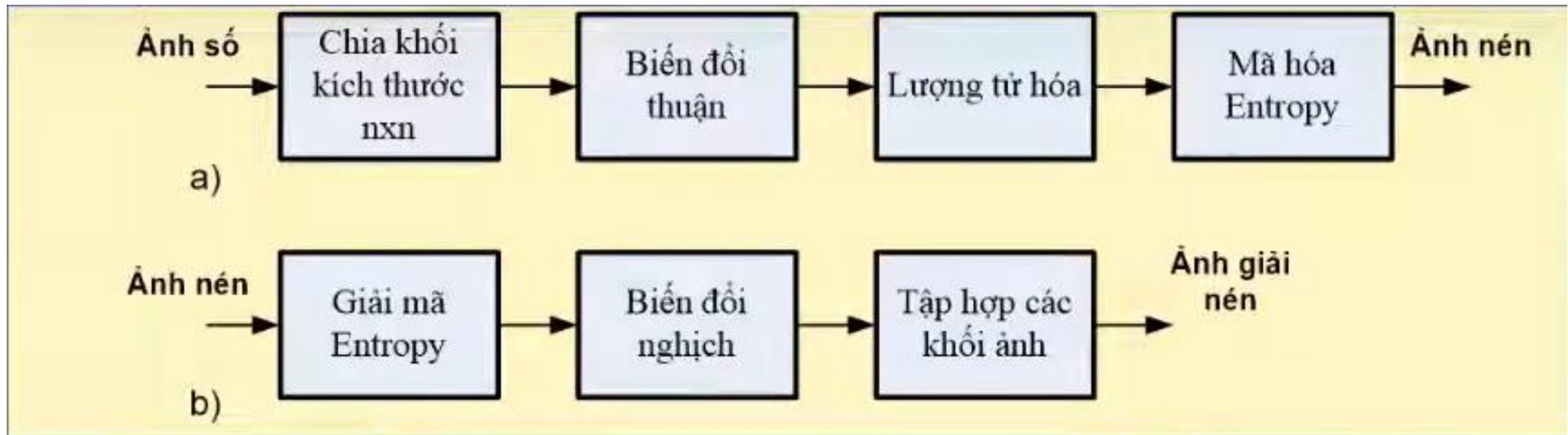
- ❑ Trong một khối ảnh đủ **nhỏ** (ví dụ 8x8) thì thành phần tần số thấp sẽ chứa nhiều thông tin hơn do cường độ xám ít thay đổi
- ❑ Mắt người khó phát hiện mất mát đối với vùng tần số cao hơn so với vùng tần số thấp



Quy trình nén ảnh jpeg

DCT thuận

lossless
compression

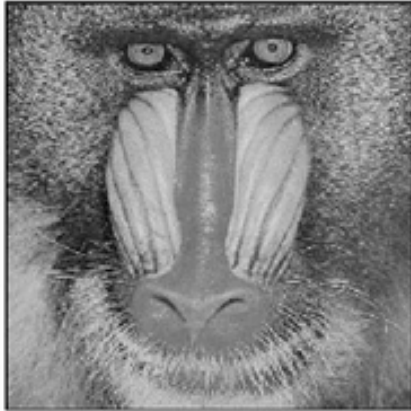


DCT nghịch

JPEG compression



Input

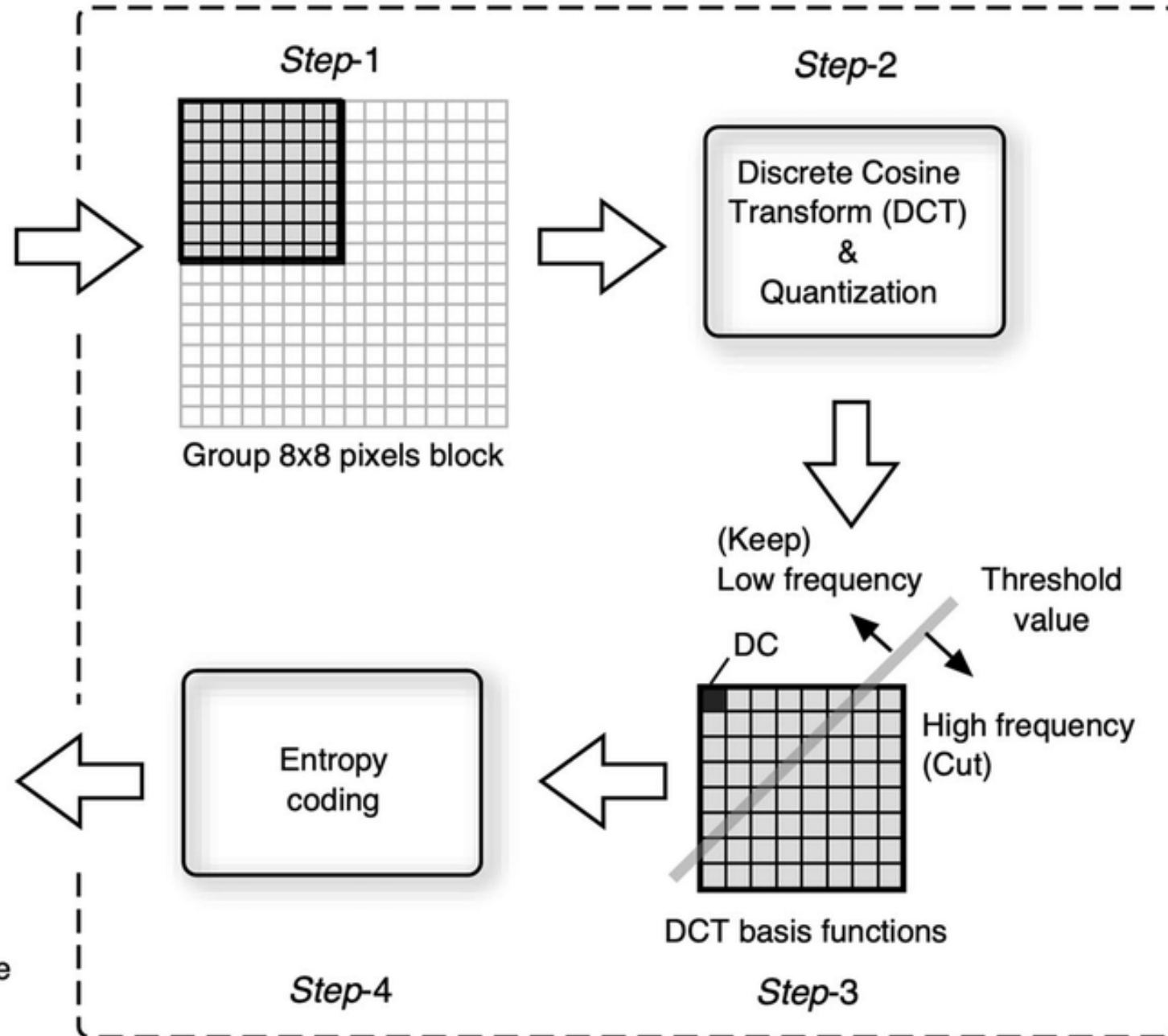


Original gray image
(large data size)

Output



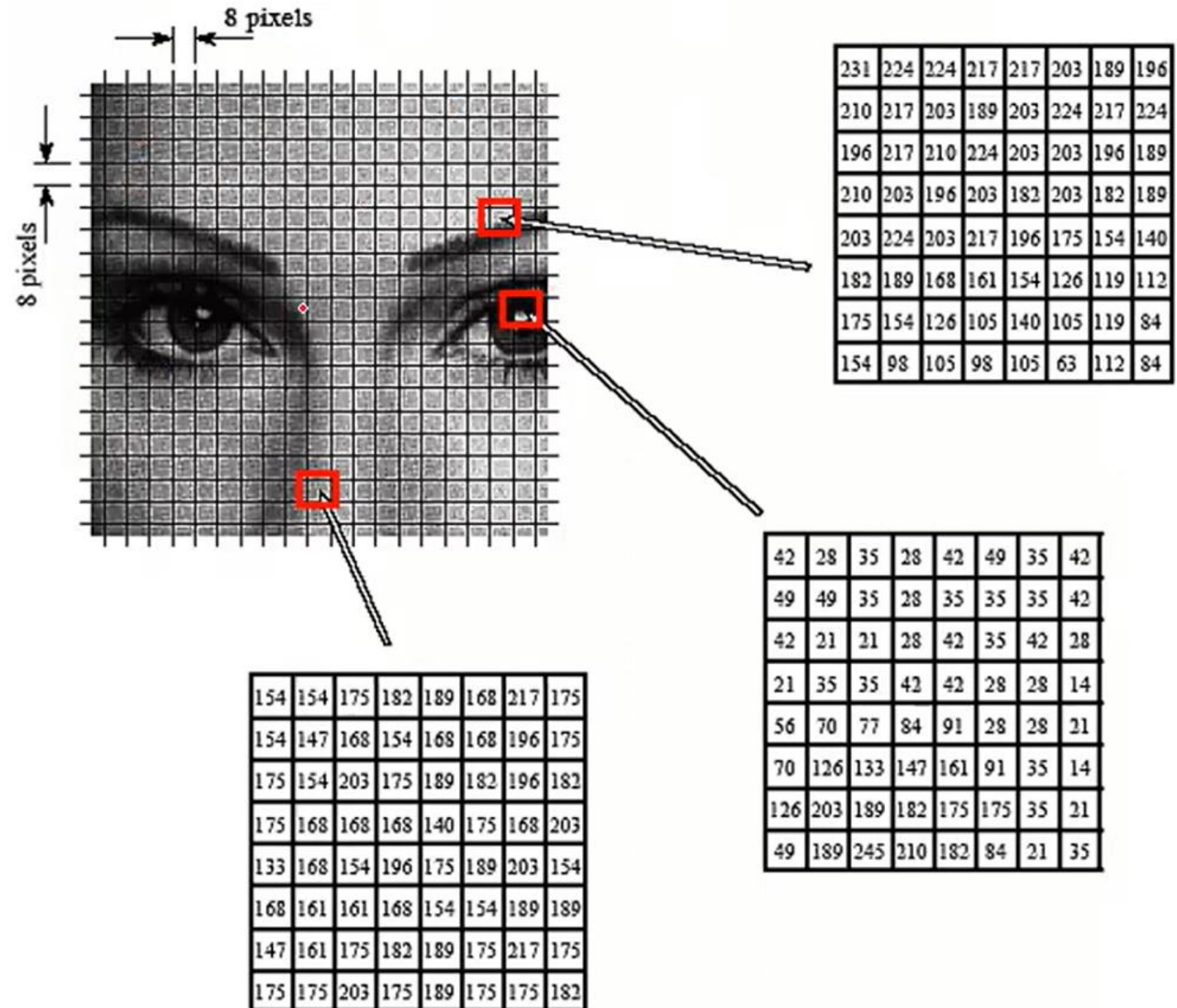
Compressed JPEG image
(small data size)



©Shien Ri (2020)

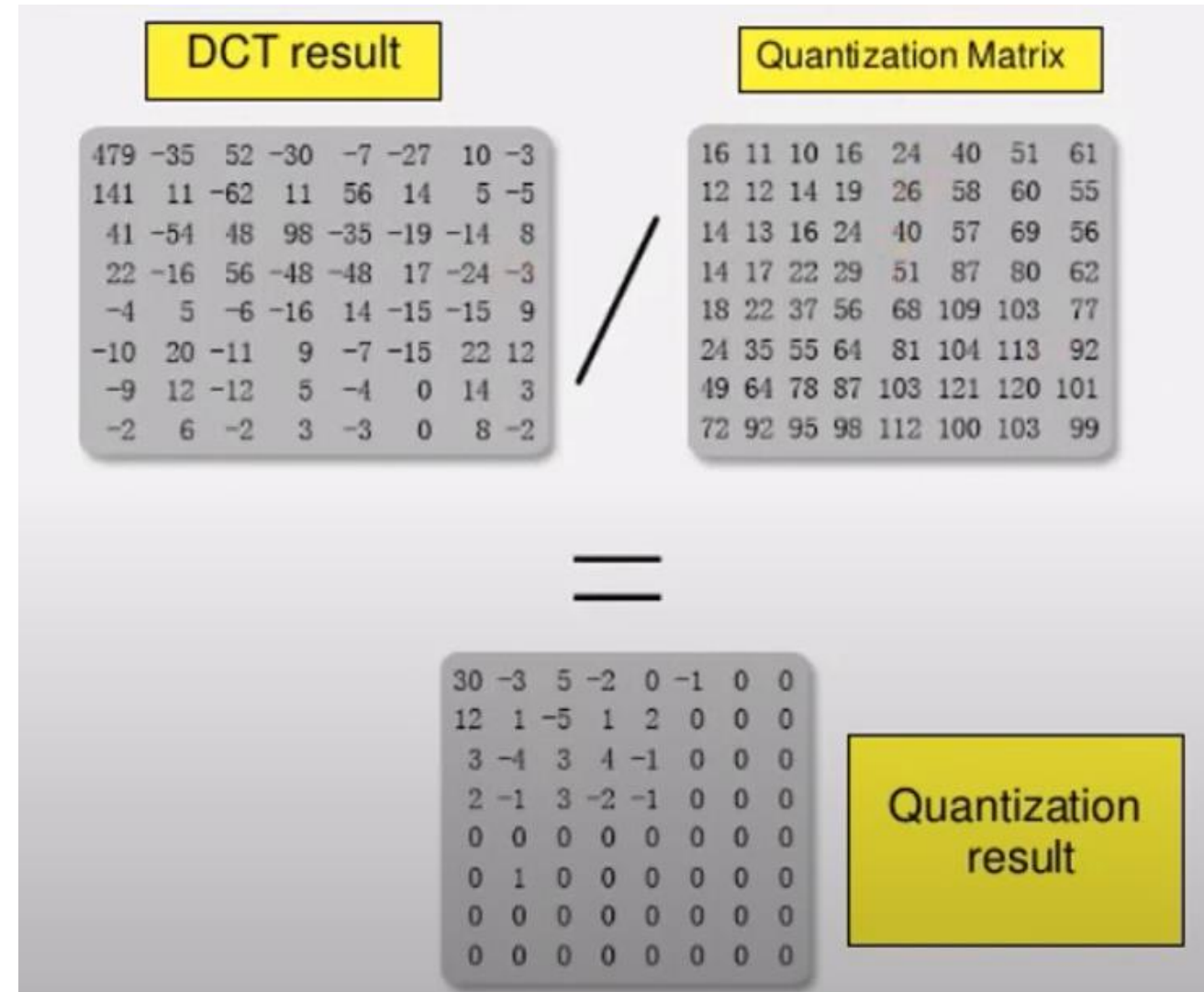
Chia ảnh

- Nếu ảnh kích thước nhỏ có thể chia thành các khối 8x8



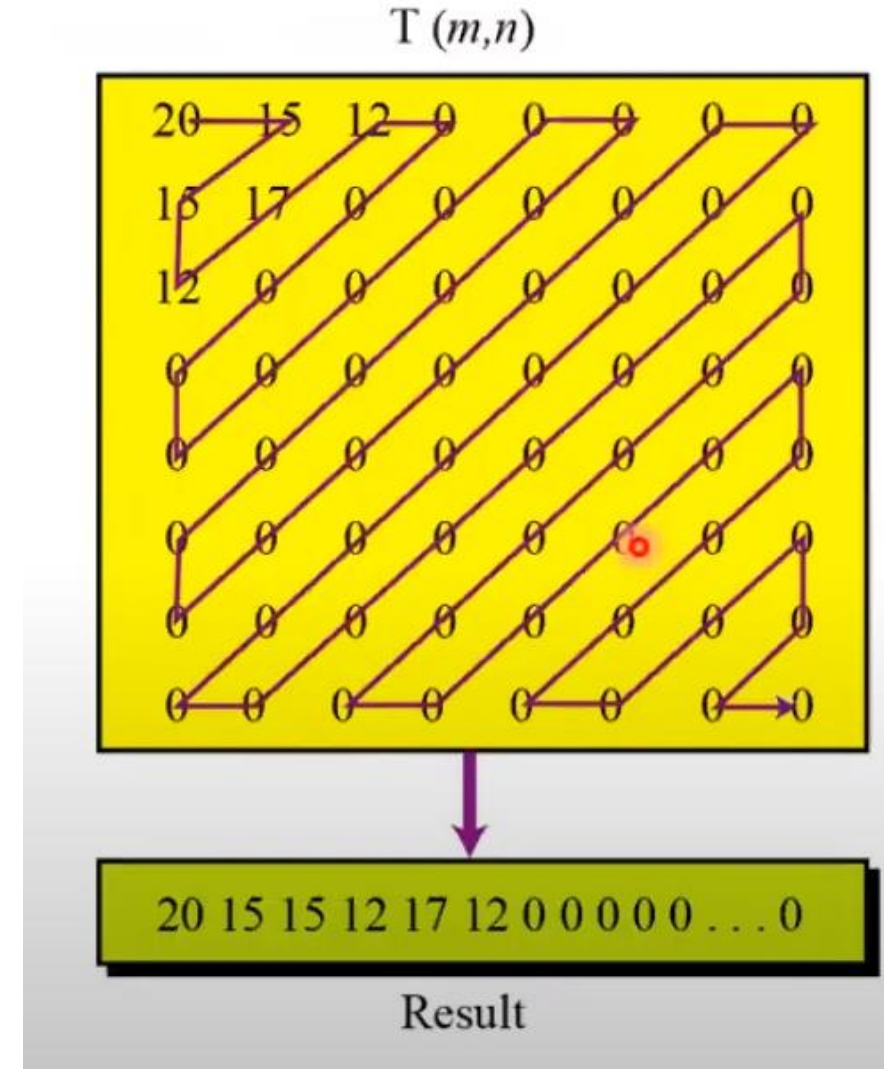
DCT thuận và Lượng tử hóa

- ❑ Sau khi chia khối 8x8 và biến đổi DCT thuận, ma trận DCT sẽ được lượng tử hóa (quantization)



Đọc zig-zag và mã hóa

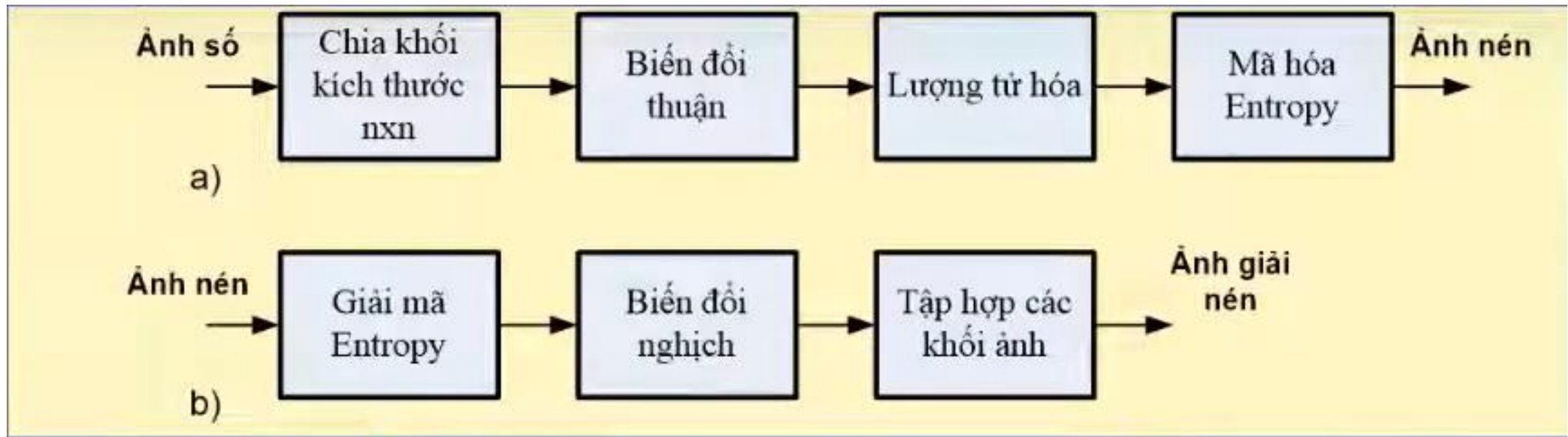
- ❑ Sau khi lượng tử hóa, ma trận T sẽ được đọc theo kiểu zig-zag để tối ưu hóa việc mã hóa Run length code (gom nhiều số 0)
- ❑ Mã hóa entropy



Quy trình nén ảnh jpeg

DCT thuận

lossless
compression



DCT nghịch

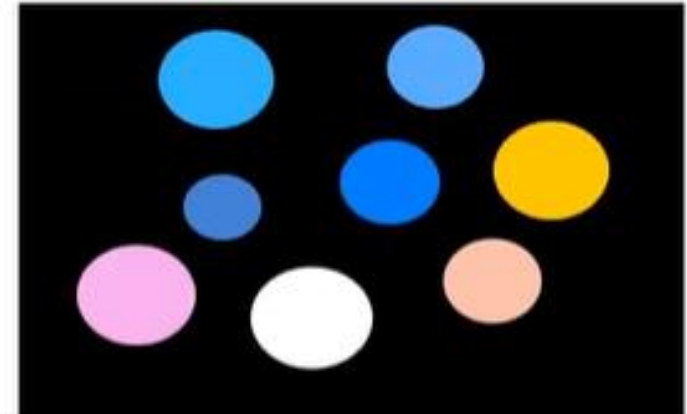
❑ Sau đó ảnh nén sẽ được giải nén

PHẦN 2. Phân đoạn ảnh (Thị giác máy tính)

Mục tiêu của phân đoạn ảnh

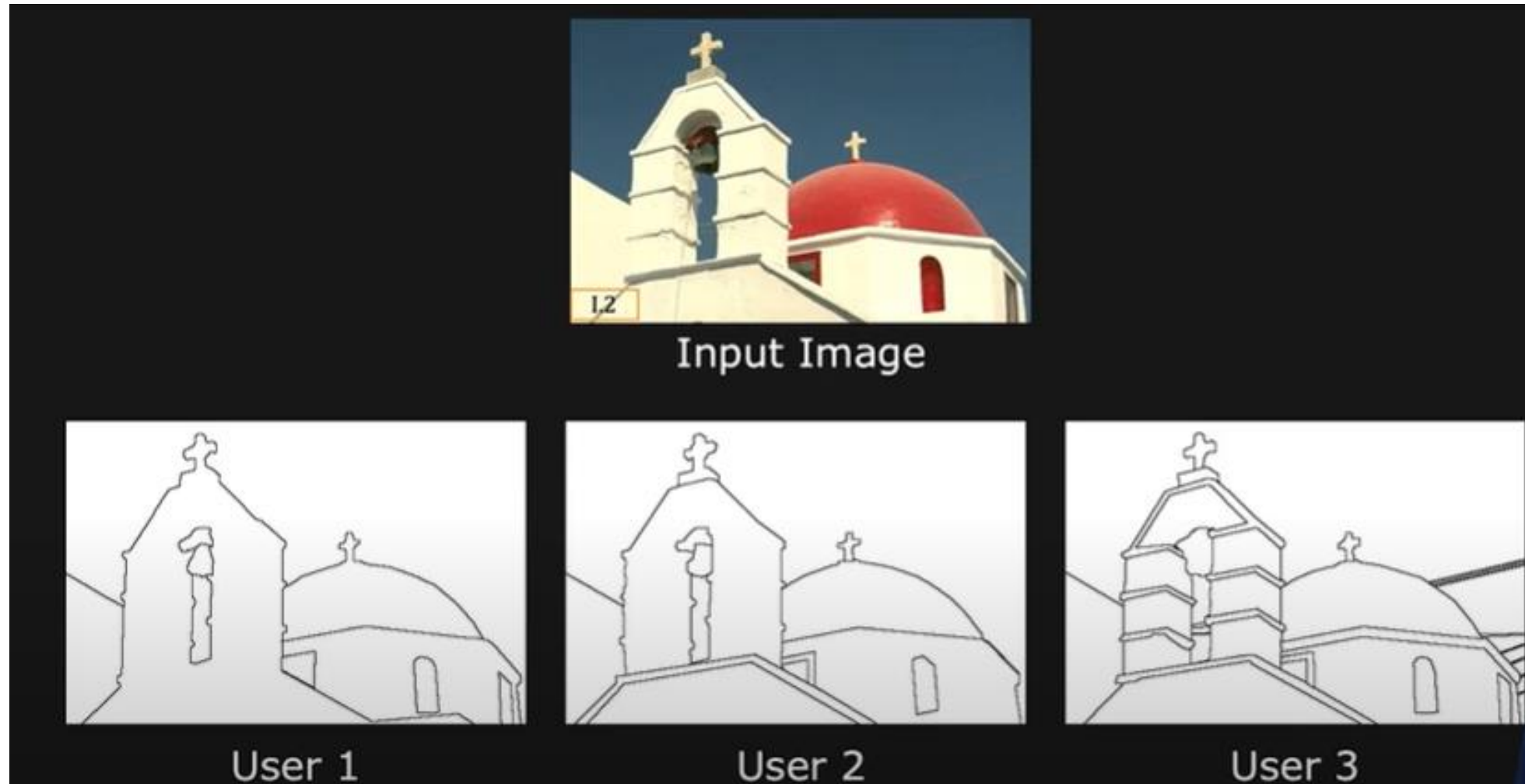
- ❑ Hiểu các đối tượng/vùng trong ảnh
- ❑ Đối sánh và nhận dạng ảnh
- ❑ Dò tìm đối tượng trong ảnh
- ❑ Phân lớp

...



1. Phân đoạn ảnh ở người

- ❑ Phân đoạn ảnh ở mắt người là chủ quan
- ❑ Rất khó để chuyển sự trực quan ở mắt người sang máy tính

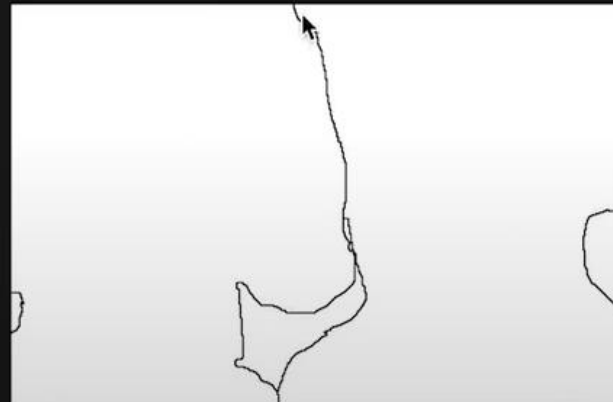


1. Phân đoạn ảnh ở người

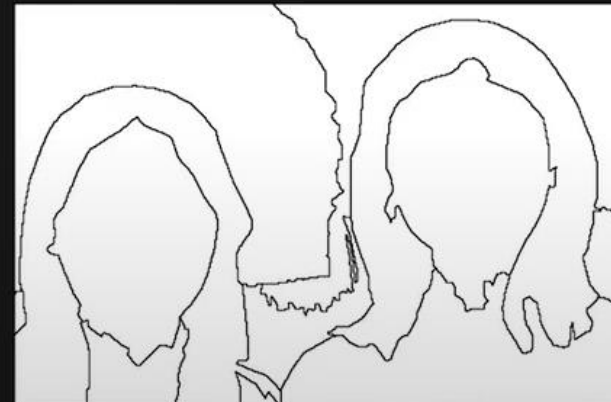
- ❑ Phân đoạn ảnh ở mặt người là chủ quan
- ❑ Rất khó để chuyển sự trực quan ở mặt người sang máy tính



Input Image



User 1



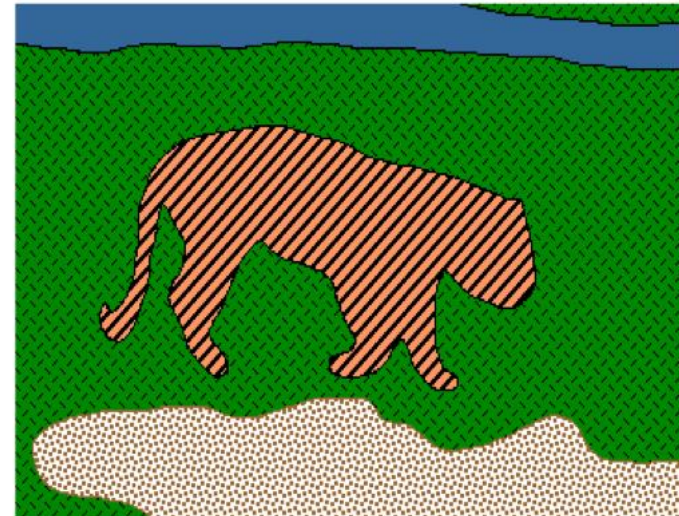
User 2



User 3

2. Phân đoạn ảnh trên máy tính

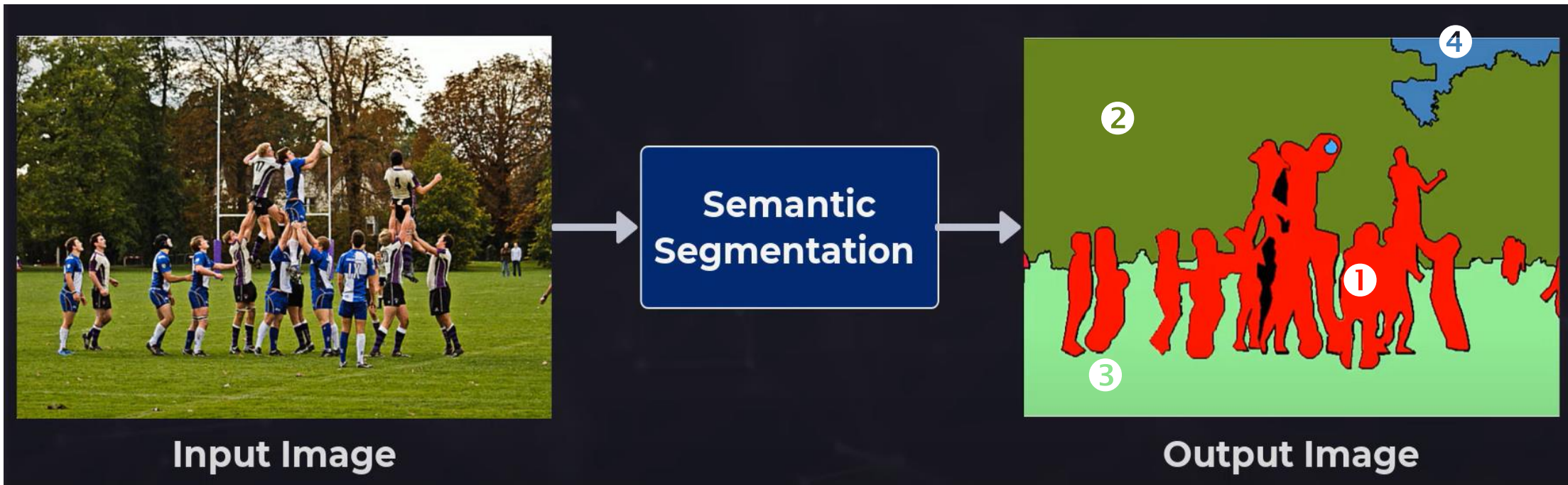
- Phân chia một ảnh (R) thành các vùng con chứa các pixel dựa trên một **tiêu chí** nào đó $R_1, R_2, R_3, \dots, R_n$ với các tính chất:
 - $R_1 \cup R_2 \cup R_3 \cup \dots \cup R_n = R$
 - $R_i \cap R_j = \emptyset$ với $i \neq j$



*Tiêu chí: cường độ, histogram, mean...

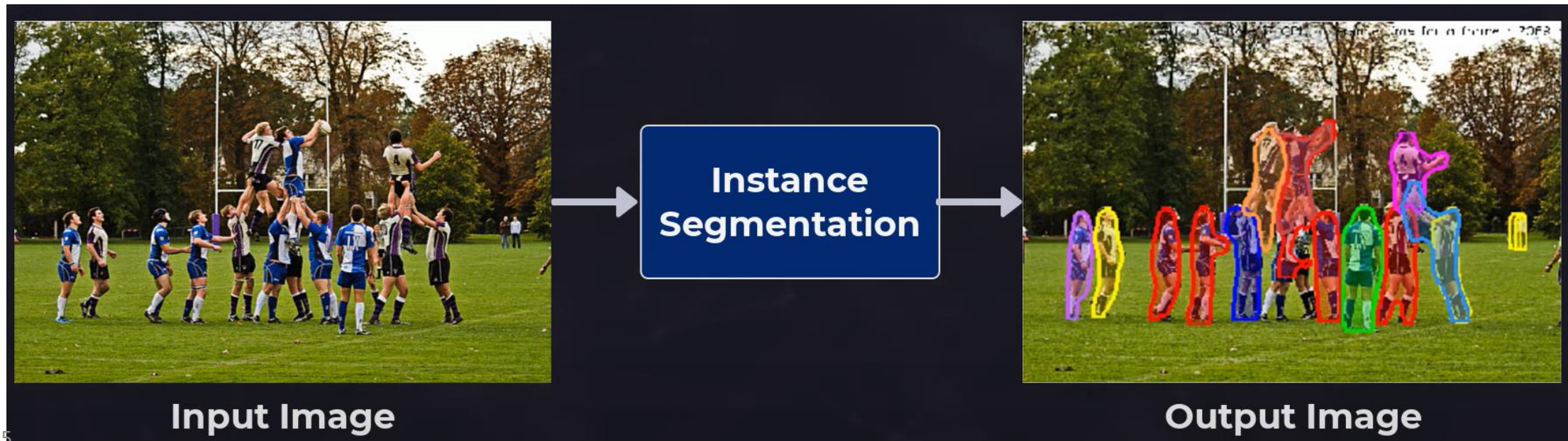
2.1 Phân đoạn theo ngữ nghĩa

- ❑ Phân đoạn theo ngữ nghĩa (semantic segmentation) dùng để gán nhãn cho từng lớp như xe hơi, người, chó, mèo... cho **mỗi pixel của ảnh**



2.2 Phân đoạn theo đối tượng cụ thể

- ❑ Phân đoạn theo đối tượng cụ thể (instance segmentation) sử dụng mask để phân biệt từng đối tượng cụ thể chung một lớp
- ❑ Quan tâm đến **boundary** của đối tượng cụ thể



3. Các phương pháp phân đoạn ảnh

